

UART HAL 接口使用指南

文档版本
发布日期

V1.1
2024-01-15

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档详细介绍了 UART 在 HAL 层的常用接口及相关数据结构，内容涵盖了数据结构、接口函数、驱动调用流程和常见问题及解决方法。本文档适用于 UWS6130E、UWS6131E、UWS6133、UWS6133E、UWS6135E 芯片平台。

读者对象

本文档主要适用于需要使用 UART 在 HAL 层中提供对外接口及相关数据结构的软件开发、测试人员。

缩略语

缩略语	英文全名	中文解释
APB	Advanced Peripheral Bus	先进外围总线
CTS	Clear to Send	允许发送
DMA	Dynamic Multithreading Architecture	动态多线程结构
FIFO	First In First Out	先进先出
HAL	Hardware Abstract Layer	硬件抽象层
RTS	Request To Send	请求发送
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发传输器

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。

符号	说明
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.1	2024-01-15	- 增加 4 驱动调用流程和 5 常见问题。 - 更新 API 函数。 - 优化完善文档内容。
V1.0	2020-01-03	第一次正式发布

关键字

UART、HAL、API

目 录

1 概述.....	1
2 数据结构.....	2
2.1 宏.....	2
2.2 枚举类型.....	2
2.2.1 uart_event.....	2
2.3 结构体.....	3
2.3.1 uart.....	3
2.3.2 uart_port.....	3
2.3.3 sprd_uart.....	4
2.3.4 uart_ops.....	6
2.3.5 termios.....	7
2.3.6 UART_INIT_PARA_T.....	9
3 API 函数.....	11
3.1 uart_port_open.....	11
3.2 uart_port_param_init.....	11
3.3 uart_set_termios.....	12
3.4 uart_write.....	12
3.5 uart_read.....	13
3.6 uart_port_close.....	13
3.7 uart_get_tfifo_len.....	14
3.8 uart_get_rfifo_len.....	14
3.9 uart_start_tx.....	15
3.10 uart_stop_tx.....	15
3.11 uart_start_rx.....	16
3.12 uart_stop_rx.....	16
3.13 uart_dump_reg.....	17
4 驱动调用流程.....	18
4.1 调用步骤.....	18
4.2 调用示例.....	18
5 常见问题.....	20
5.1 UART 中断过多导致 Kernel Crash.....	20
5.2 UART 输出乱码.....	20
5.3 UART 无法正常发送数据.....	21

图目录

图 1-1 UART HAL 在系统框架中的位置示意图.....	1
----------------------------------	---

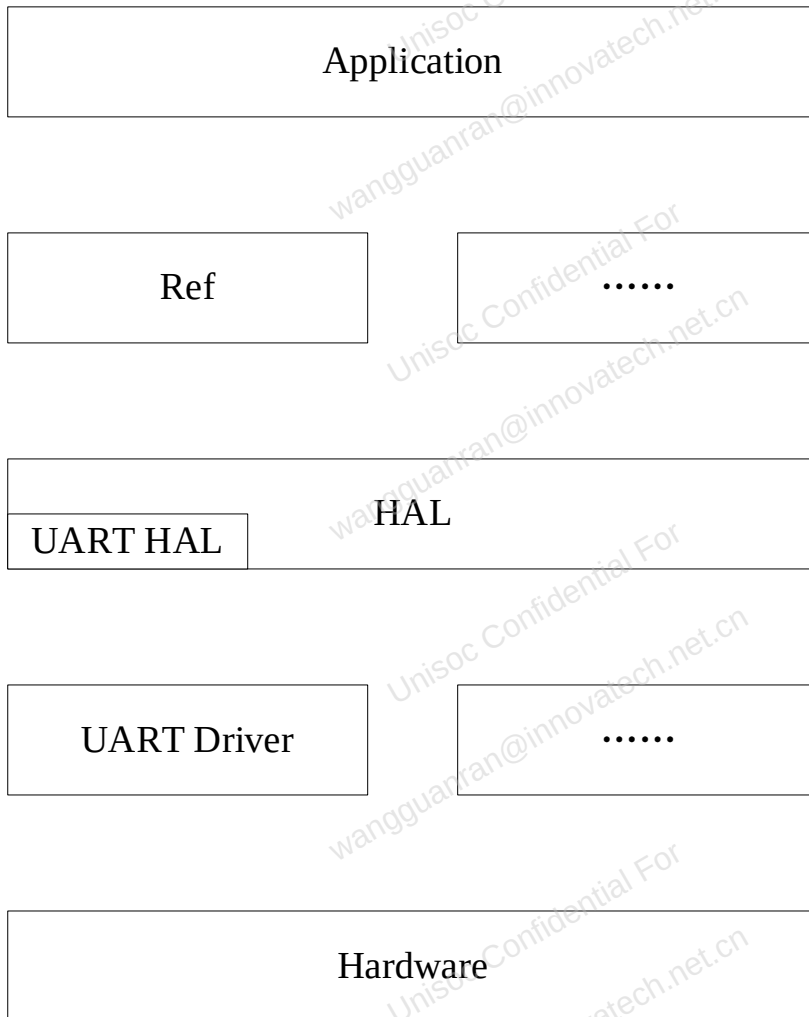
表目录

表 2-1 bit[7:0]定义	8
表 5-1 UART 流控性能参数.....	21

1 概述

UART（Universal Asynchronous Receiver/Transmitter，通用异步收发传输器）HAL（Hardware Abstract Layer，硬件抽象层）在系统框架中的位置示意图如图 1-1 所示。

图1-1 UART HAL 在系统框架中的位置示意图



HAL 层提供了 UART 所有的对外接口，上层调用 HAL 层中 UART 相关接口来配置 UART 控制器，控制 UART 完成相关操作。

2 数据结构

本章主要介绍 UART 相关的数据结构，包括宏、枚举类型及结构体。

2.1 宏

宏名	说明
UART_DEBUG	调用 <code>uart_dump_reg</code> 来获取 UART 寄存器信息。

2.2 枚举类型

2.2.1 uart_event

【功能】

定义 UART 读、写两种事件类型，通过传递相关事件告知使用者 UART 当前的状态。

【定义】

```
enum uart_event {  
    EVENT_READ,  
    EVENT_WRITE  
};
```

【成员说明】

成员	说明
EVENT_READ	指示 UART 执行读操作。
EVENT_WRITE	指示 UART 执行写操作。

2.3 结构体

2.3.1 uart

【功能】

定义 uart_port 结构体以及最多支持的 UART 个数。

【定义】

```
struct uart {  
    struct uart_port *up;  
    int count;  
};
```

【成员说明】

成员	说明
up	uart_port 结构体指针。
count	最多支持的 UART 个数。

2.3.2 uart_port

【功能】

定义配置 UART 参数的结构体（sprd_uart）及 UART 操作函数等相关内容。

【定义】

```
struct uart_port {  
    struct sprd_uart *su;  
    struct uart_ops *ops;  
    struct termios termios;  
    BOOLEAN suspend;  
    BOOLEAN init;  
};
```

【成员说明】

成员	说明
su	sprd_uart 结构体指针。

成员	说明
ops	UART 操作函数指针。
termios	UART 参数配置。
suspend	UART 挂起状态。 • TRUE: UART 挂起。 • FALSE: UART 未挂起。
init	UART 初始化状态。 • TRUE: UART 初始化完成。 • FALSE: UART 初始化未完成。

2.3.3 sprd_uart

【功能】

定义 UART 的配置参数。

【定义】

```

struct sprd_uart {
    int index;
    uint32 base;
    int irq;

    /*apb eb*/
    uint32 apb_base;
    uint32 eb;
    uint32 reg_offset;

    /* source clock config */
    uint32 sclk;
    uint32 sclk_base;
    uint32 sclk_bit;

    /* dma mode */
    uint32 dma_mode;
    uint32 tx_id;
    uint32 rx_id;
    
```

```

uint32 baud;

UART_CB cb;
uint32 tx_chn;
uint32 rx_chn;

int irq_cnt;
int write_cnt;
};
  
```

【成员说明】

成员	说明
index	UART 编号。
base	UART 寄存器基地址。
irq	UART 中断号。
apb_base	使能 UART 开关的 APB（Advanced Peripheral Bus，先进外围总线）地址。
eb	UART APB 使能位。
reg_offset	UART 偏移地址。
sclk	UART 时钟源。
sclk_base	UART 时钟源选择地址。
sclk_bit	UART 时钟源选择位。
dma_mode	DMA（Dynamic Multithreading Architecture，动态多线程结构）传输模式。 • 0 代表非 DMA 模式 • 1 代表 DMA 模式。
tx_id	UART TX DMA 请求号。
rx_id	UART RX DMA 请求号。
baud	UART 波特率，单位：bit/s。
cb	UART 回调函数。
tx_chn	UART TX DMA 通道号。
rx_chn	UART RX DMA 通道号。
irq_cnt	UART 中断计数。
write_cnt	UART 写计数。

2.3.4 uart_ops

【功能】

定义 UART 的操作函数。

【定义】

```
struct uart_ops {
    int (*lower_init)(struct sprd_uart *, struct termios *);
    int (*init)(struct sprd_uart *, UART_CB);
    int (*shutdown)(struct sprd_uart *);
    int (*set_termios)(struct sprd_uart *, struct termios *);
    int (*get_termios)(struct sprd_uart *, struct termios *);
    uint32 (*tx)(struct sprd_uart *, uint8 *, uint32);
    uint32 (*rx)(struct sprd_uart *, uint8 *, uint32);
    int (*get_tfifo_len)(struct sprd_uart *);
    int (*get_rfifo_len)(struct sprd_uart *);
    int (*start_tx)(struct sprd_uart *);
    int (*stop_tx)(struct sprd_uart *);
    int (*start_rx)(struct sprd_uart *);
    int (*stop_rx)(struct sprd_uart *);
    int (*apb_enable)(struct sprd_uart *);
};
```

【成员说明】

成员	说明
lower_init	UART 基础功能配置初始化函数指针。
init	UART 全功能初始化函数指针。
shutdown	UART 关闭函数指针。
set_termios	UART 参数配置函数指针。
get_termios	UART 参数获取函数指针。
tx	UART 写函数指针。
rx	UART 读函数指针。
get_tfifo_len	UART 用于获取保存在 TX FIFO（First In First Out，先进先出）的数据个数的函数指针。
get_rfifo_len	UART 用于获取保存在 RX FIFO 的数据个数的函数指针。

成员	说明
start_tx	UART 开启 TX 中断函数指针。
stop_tx	UART 关闭 TX 中断函数指针。
start_rx	UART 开启 RX 中断函数指针。
stop_rx	UART 关闭 RX 中断函数指针。
apb_enable	UART APB 使能开关函数指针。

2.3.5 termios

【功能】

定义初始化 UART 的波特率以及数据位、停止位、奇偶校验位、硬件流控和 DMA 模式等参数。

【定义】

```
struct termios {
    /* controller flag, the low 8 bit for baudrate */
    uint32 baud;
    /* enable dma mode, hw flow ctrl */
    uint32 cflag;
};
```

【成员说明】

成员	说明
baud	UART 波特率，单位：bit/s。
cflag	用于 UART 数据位、停止位、奇偶校验位、硬件流控和 DMA 模式等参数配置，cflag 有 32bit，目前只用到低 8 位，其他保留。bit[7:0]定义如表 2-1 所示。cflag 中 bit[7:0]相关的宏定义如下： <pre> #define C_DMA 0x00000001 //UART使能DMA模式 #define CRX_RTS 0x00000002 //开启接收硬件流控 #define CTX_CTS 0x00000004 //开启发送硬件流控 #define CSIZE 0x00000018 //数据位控制bit，默认为8位 #define CS5 0x00000000 //5个数据位 #define CS6 0x00000008 //6个数据位 #define CS7 0x00000010 //7个数据位 #define CS8 0x00000018 //8个数据位 </pre>

成员	说明
	<pre>#define CSSTOPB 0x00000020 //2位停止位</pre> <pre>#define PAREN_DIS 0x00000000 //无校验位</pre> <pre>#define PARENB 0x00000040 //校验使能位</pre> <pre>#define PAREVEN 0X00000000 // 偶校验，默认值</pre> <pre>#define PARODD 0x00000080 //奇校验</pre>

表2-1 bit[7:0]定义

BIT 位	含义
BIT0	配置 DMA 传输模式。 0 表示不开启 DMA 传输。 1 表示开启 DMA 传输。
BIT1	配置接收硬件流控。 0 表示不开启接收硬件流控。 1 表示开启接收硬件流控。
BIT2	配置发送硬件流控。 0 表示不开启发送硬件流控。 1 表示开启发送硬件流控。
BIT[4:3]	配置数据位宽度。 00 表示数据位配置为 5 位。 01 表示数据位配置为 6 位。 10 表示数据位配置为 7 位。 11 表示数据位配置为 8 位。
BIT5	配置停止位。 0 表示一个停止位。 1 表示两个停止位。
BIT6	配置奇偶校验使能位。 0 表示不使能校验。 1 表示使能校验。
BIT7	配置奇偶校验位。 0 表示偶校验。 1 表示奇校验。

【配置示例】

当 UART 不开启 DMA 传输，无硬件流控、8 位数据位，1 个停止位，无校验位时，配置 cflag:

```
cflag |= CS8 | PAREN_DIS
```

2.3.6 UART_INIT_PARA_T

【功能】

定义动态修改 UART 的参数。

【定义】

```
typedef struct
{
    uint16 tx_watermark;
    uint16 rx_watermark;
    uint32 baud_rate;
    UART_PARITY_E      parity;
    UART_PARITY_SET_E  parity_enable;
    UART_BYTE_SIZE_E   byte_size;
    UART_STOP_BIT_E    stop_bits;
    UART_FLOW_CTRL_E   flow_control;
    UART_DS_CTRL_E     ds_wakeup_en;
    UART_DS_TYPE_E     ds_wakeup_type;
    UART_MODE_E dma_mode;
} UART_INIT_PARA_T;
```

【成员说明】

成员	说明
tx_watermark	TX FIFO 空阈值。
rx_watermark	RX FIFO 满阈值。
baud_rate	波特率，单位：bit/s。
parity	校验位。
parity_enable	校验使能位。
byte_siz	数据位。
stop_bits	停止位。
flow_control	流控配置。

成员	说明
ds_wakeup_en	唤醒使能。
ds_wakeup_type	唤醒模式选择。
dma_mode	DMA 功能配置。

3 API 函数

本章介绍 HAL 层 UART 常用对外接口函数，对接口函数功能进行详细描述。

3.1 uart_port_open

【函数功能】

该函数用于开启 UART 控制器，初始化 UART、注册中断及回调函数等。

【函数原型】

```
PUBLIC int uart_port_open(int n, UART_CB cb);
```

【参数说明】

参数名称	含义
n	UART 编号。
cb	UART 回调函数，通过传入一个回调函数后，当 UART 中断检测到读/写事件之后会通过一个回调函数将相关的事件传递给上层，从而上层能够明确当前 UART 的状态，并进行下一步动作。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.2 uart_port_param_init

【函数功能】

该函数用于驱动初始化时，识别 UART 控制器的相关参数，包括波特率，奇偶校验，流控等，并将参数赋值给 termios 结构体，调用 uart_set_termios 最终实现参数配置。

【函数原型】

```
PUBLIC int uart_port_param_init(int n, UART_INIT_PARAM_T *param);
```

【参数说明】

参数名称	含义
n	UART 编号。
param	UART 参数配置结构体指针。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.3 uart_set_termios

【函数功能】

该函数用于配置 UART 控制器的相关参数，包括波特率、奇偶校验、流控等。

【函数原型】

```
PUBLIC int uart_set_termios(uint32 n, struct termios *termios);
```

【参数说明】

参数名称	含义
n	UART 编号。
termios	UART 参数配置结构体指针。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.4 uart_write

【函数功能】

该函数用于写 UART 数据。

【函数原型】

```
PUBLIC uint32 uart_write(uint32 n, const uint8 *buf, uint32 len);
```

【参数说明】

参数名称	含义
n	UART 编号。
buf	需要发送数据的缓存 buffer 指针。
len	需要发送数据的长度，单位：Byte。

【返回值】

- 操作成功：已发送数据个数。
- 操作失败：0。

3.5 uart_read

【函数功能】

该函数用于读取 UART 数据。

【函数原型】

```
PUBLIC uint32 uart_read(uint32 n,uint8 *buf, uint32 len);
```

【参数说明】

参数名称	含义
n	UART 编号。
buf	需要接收数据的缓存 buffer 指针。
len	需要接收数据的长度，单位：Byte。

【返回值】

- 操作成功：已读取数据个数。
- 操作失败：0。

3.6 uart_port_close

【函数功能】

该函数用于关闭 UART。

【函数原型】

```
PUBLIC void uart_port_close(int n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

无。

3.7 uart_get_tfifo_len

【函数功能】

该函数用于获取 UART TX FIFO 缓存未发出的数据个数。

【函数原型】

```
PUBLIC uint32 uart_get_tfifo_len(uint32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：UART TX FIFO 缓存未发出的数据个数。
- 操作失败：0。

3.8 uart_get_rfifo_len

【函数功能】

该函数用于获取 UART RX FIFO 缓存未读取的数据个数。

【函数原型】

```
PUBLIC uint32 uart_get_rfifo_len(uint32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：UART RX FIFO 缓存未读取的数据个数。
- 操作失败：0。

3.9 uart_start_tx

【函数功能】

该函数用于开启 UART TX 中断。

【函数原型】

```
PUBLIC int uart_start_tx(uint32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.10 uart_stop_tx

【函数功能】

该函数用于关闭 UART TX 中断。

【函数原型】

```
PUBLIC uint32 uart_stop_tx (uint32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.11 uart_start_rx

【函数功能】

该函数用于开启 UART RX 中断。

【函数原型】

```
PUBLIC uint32 uart_start_rx (uint32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.12 uart_stop_rx

【函数功能】

该函数用于关闭 UART RX 中断。

【函数原型】

```
PUBLIC uint32 uart_stop_rx (uint32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：0。
- 操作失败：负数。

3.13 uart_dump_reg

【函数功能】

该函数用于获取 UART 寄存器信息。

【函数原型】

```
int uart_dump_reg(unit32 n);
```

【参数说明】

参数名称	含义
n	UART 编号。

【返回值】

- 操作成功：0。
- 操作失败：负数。

【注意事项】

此接口需要开启 UART_DEBUG 宏方可使用。

4 驱动调用流程

本章节主要介绍调用 UART 正常工作的软件流程。

4.1 调用步骤

- 步骤 1 调用 `uart_port_open` 接口初始化 UART 并传递回调函数。
- 步骤 2 调用 `uart_port_param_init` 或者 `uart_set_termios` 配置 UART 波特率、空满阈值、帧格式、传输模式等参数。
- 步骤 3 调用 `uart_read/uart_write` 接口函数读/写数据。
- 步骤 4 调用 `uart_port_close` 接口函数关闭相应的 UART 端口。

----结束

4.2 调用示例

本节以“配置 UART 参数并通过 UART 写数据”为例说明 UART 调用流程。

```
void uart_callback(unsigned int evt)
{
    /* nothing to do */
}

void uart_test(uint32 port)
{
    char buf[] = "debug uart write"
    UART_INIT_PARAM param;
    uart_port_open(port, uart_callback);
    param.baud_rate = 115200;
    param.stop_bits = ONE_STOP_BIT;
    param.parity_enable = PARITY_DISABLE;
    param.byte_size = EIGHT_BITS;
    param.flow_control = NO_FLOW_CONTROL;
    param.dma_mode = UART_DMA_MODE_MAX;
    param.ds_wakeup_en = DS_WAKEUP_DISABLE;
```

```
uart_port_param_init(port, &param);  
uart_write(port, buf, sizeof(buf));  
uart_port_close(port);  
}
```

5 常见问题

5.1 UART 中断过多导致 Kernel Crash

现象描述

UART 中断过多，每秒中断次数在 1000 次以上。

可能原因

通常是由于 UART 的 RX 引脚被外部持续拉低，进而产生大量的 break 中断。

定位思路及解决方法

需要确保 RX 为上拉状态。

5.2 UART 输出乱码

现象描述

UART 端口输出乱码。

可能原因

UART 输出乱码一般是波特率设置错误导致的。

定位思路及解决方法

1. 检查通信两端的串口波特率配置是否一致。
2. 检查 pin 的上下拉配置是否正确，一般作为 UART 功能的 pin 需要使能上拉功能。
3. 检查硬件，避免外部干扰引入。

说明

若有疑问请联系展锐 FAE 协助处理。

5.3 UART 无法正常发送数据

现象描述

UART 无法正常发送数据。

可能原因

一般 UART 不能发送数据的原因是开启了硬件流控功能，当 CTS（Clear to Send，允许发送）引脚被远端的 RTS（Request To Send，请求发送）引脚拉高后就不会进行数据的传输。

UART 流控性能参数请参见表 5-1。

表5-1 UART 流控性能参数

输入时钟频率（MHz）	最大支持速度（bit/s）
26	- 带流控：3250000 - 不带流控：115200
48	- 带流控：4000000 - 不带流控：115200
64	- 带流控：4000000 - 不带流控：115200
51.2	- 带流控：4000000 - 不带流控：115200
96	- 带流控：4000000 - 不带流控：115200

定位思路及解决方法

1. 检查 pin 的配置是否正常。
2. 检查远端的设备是否有持续接收数据，远端设备没有持续接收数据会导致传输的停止。

说明

若有疑问请联系展锐 FAE 协助处理。